



Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py`.
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent` .
2. Import required structures: `from collections import deque` and `import heapq` .
3. **BFS:** Create `def bfs_search(...)` . Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)` . Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)` . Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__` , add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'` .
2. In `sense_and_act(self, percept)` , check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']` , execute the search method matching `self.active_algo` , and store the resulting sequence of actions in `self.plan` .
4. Return the first action from the plan using `return self.plan.pop(0)` .
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

- **State Space:** Represents the physical configuration and topological graph of the environment (the physical (x, y) coordinate grid, wall boundaries, and valid transitions between adjacent cells). It is finite, fixed, and contains loops and bidirectional pathways.
- **Search Tree:** A dynamic directed data structure constructed by the agent during its offline planning phase to simulate paths from the start state. Paths unfold from the root node downward; unlike the underlying physical graph, a single physical state can appear multiple times across different branches of the search tree unless duplicate detection is applied.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

- **Problem Solved:** The `reached` set acts as a closed list that tracks already discovered and expanded states. It converts a naive **Tree Search** into an efficient **Graph Search** by preventing the redundant re-expansion of previously visited states along redundant or cyclical paths.
- **Consequences for DFS:** Without a `reached` set, a Depth-First Search agent operating in a reversible grid environment will fall into an **infinite loop** (e.g., oscillating back and forth between two adjacent empty cells such as $(0,0) \rightarrow (1,0)$). The call stack/frontier expands continuously until it triggers an `OutOfMemory` or recursion depth error.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

- **BFS Optimality:** BFS expands the search frontier level-by-level (shallowest nodes first). Because every step on the grid has an equal transition cost ($g = 1$), BFS evaluates all candidate paths of length k before evaluating any path of length $k + 1$. Thus, the first time a goal node is selected for expansion, the discovered path is guaranteed to have the minimum number of steps (optimal shortest path).
- **DFS Suboptimality:** DFS explores each path to its deepest possible leaf node before backtracking via a LIFO stack. It does not incorporate path cost into its decision-making, so it commits to long, convoluted corridors and dead ends,

leading to wandering, erratic, and highly suboptimal routes even when the target is only a few cells away.

4. (Evaluate) If we scaled `visual_grid_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

- **BFS / UCS Memory Bottleneck:** BFS stores the entire frontier in RAM simultaneously, leading to an exponential space complexity of $\mathcal{O}(b^d)$ (where b is the branching factor ≈ 4 and d is the depth/distance to food). On a 1000×1000 grid where a target may be hundreds of steps away, the frontier demands gigabytes of memory, causing the Python process to crash with an `OutOfMemory` exception.
- **DFS Space Advantage:** DFS only needs to maintain the current path from the root down to the active node, plus unexpanded sibling candidates at each level. Its space complexity is linear: $\mathcal{O}(m)$ (where m is the maximum depth). DFS consumes drastically less memory than BFS and avoids crashing the interpreter on large-scale maps, despite generating suboptimal paths.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.

